

Telemetry's long way home

A Kubernetes odyssey with Prometheus, OpenTelemetry, and Elastic — August 24, 2026

Agenda

- | | |
|----|--|
| 01 | Twelve steps between your pod and your dashboard |
| 02 | Lane 1 — a Prometheus metric |
| 03 | Lane 2 — an OpenTelemetry trace |
| 04 | The four settings that matter, and who owns them |
| 05 | Live demo — Kubernetes to Elastic |

Two lines of YAML, twelve steps you never see

```
# a pod annotation
instrumentation.opentelemetry.io/inject-java: "true"
```

```
# a PodMonitor
podMetricsEndpoints:
  - port: http
    path: /metrics
```

Between those two lines and a chart in a vendor UI sit **twelve hops**, six per signal. Every one can **drop**, **relabel**, **enrich** or **sample** what you sent.

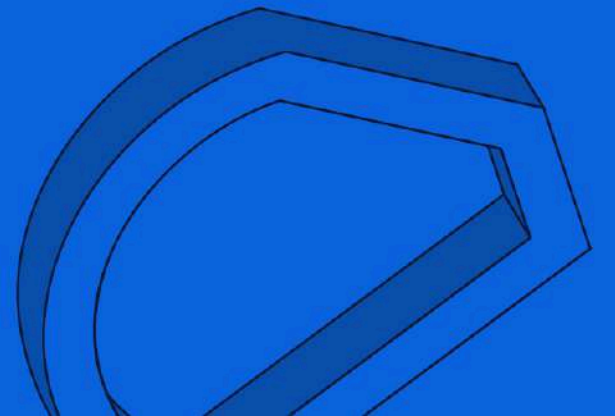
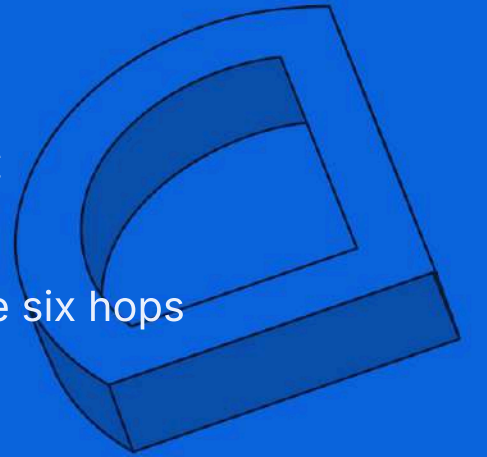
Each is a piece of configuration somebody owns: a `PodMonitor`, a `relabel_configs` block, an `Instrumentation` CR, a processor in a collector.

Most trouble here is not failure. It is **silent policy** — a component doing exactly what it was configured to do.

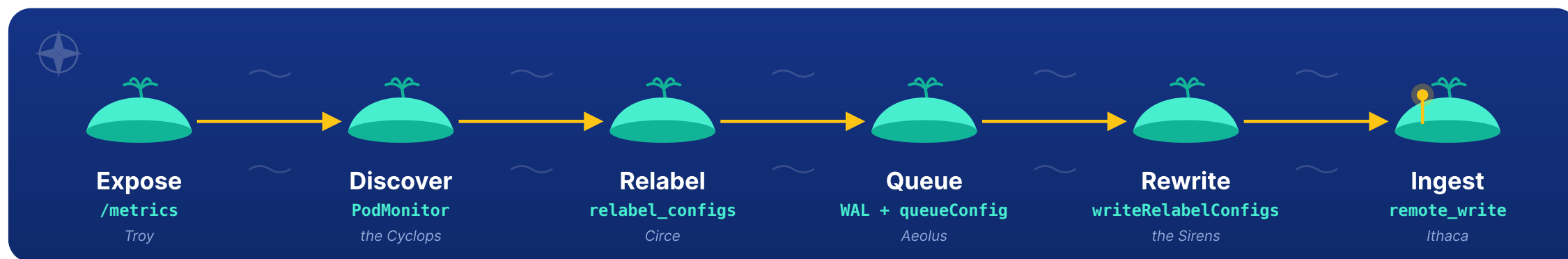
Everything in this talk runs on a laptop, in k3s, with OSS components only, and ships to a real managed endpoint. Repo at the end.

Lane 1 — a Prometheus metric

node-exporter, kube-state-metrics, kubelet/cAdvisor and your app all take the same six hops



The six steps a metric takes



Only the last hop is Elastic-specific. Change the endpoint and hops 1–5 are identical for any remote_write backend.

Hop 1 · expose · /metrics

Where metrics come from: plain text on an HTTP port



Every voyage starts as something
left lying on a beach

```
node-exporter      :9100  the node's /proc
kube-state-metrics :8080  API server object state
kubelet /metrics/cadvisor  per-container usage
your app           :8000  whatever you counted
```

prometheus.io/docs/instrumenting/exposition_formats

Nothing is "sent" at this hop. Every source is a passive HTTP endpoint that has no idea who scrapes it, or whether anyone does.

A PodMonitor tells Prometheus which pods to scrape



12 lines of PodMonitor become
part of a 1,573-line scrape config

```
# what you write – 12 lines
kind: PodMonitor
spec:
  selector:
    matchLabels: {app: checkout}
  podMetricsEndpoints:
    - port: http
      path: /metrics
```

```
# what the Operator generates
job_name: podMonitor/demo/checkout/0
kubernetes_sd_configs:
  - role: pod
    namespaces: {names: [demo]}
relabel_configs:
  - action: keep
    source_labels:
      - __meta_kubernetes_pod_label_app
    regex: (checkout);true
```

In this demo: 12 lines of PodMonitor became part of a 1,573-line generated config. Read it with `kubectl get secret prometheus-... -o jsonpath=... | gunzip`.

Relabeling decides what a metric is called



`__meta_kubernetes_*` is discarded
unless a rule copies it across

```
relabel_configs:      # BEFORE the scrape
- source_labels: [__meta_kubernetes_pod_name]
  target_label: pod
- source_labels: [__meta_kubernetes_pod_node_name]
  target_label: node
```

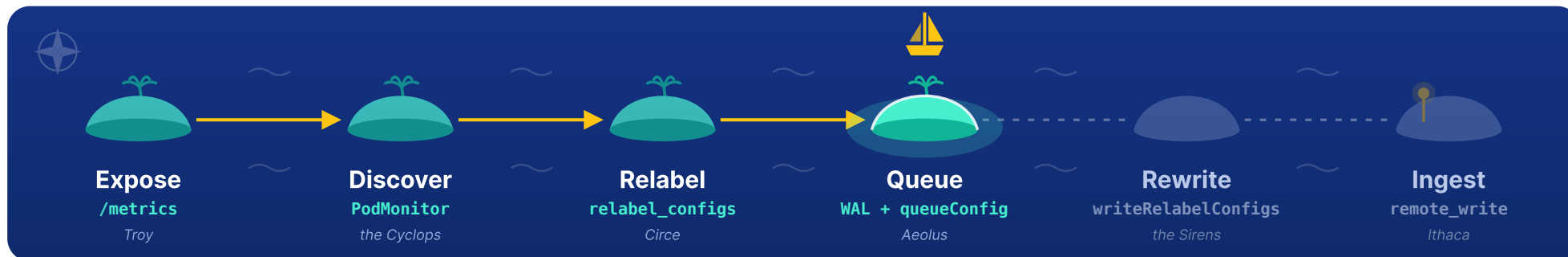
Service discovery attaches dozens of `__meta_kubernetes_*` labels. They are **all discarded** unless a rule copies them across.

`relabel_configs` decides **what to scrape and what to call it**.
`metric_relabel_configs` runs **after** the scrape and can drop series before they are ever stored.

"Which pod produced this number" is a choice made here — not something the application knows or controls.

prometheus.io/docs/prometheus/latest/configuration/configuration/#relabel_config

Prometheus writes to disk first, then sends



remote_write tails the WAL:
an outage becomes backlog

```
queueConfig:  
  capacity: 10000  
  maxShards: 10  
  minShards: 1  
  maxSamplesPerSend: 2000  
  batchSendDeadline: 5s
```

```
time()  
- prometheus_remote_storage_queue_highest_sent_timestamp_seconds
```

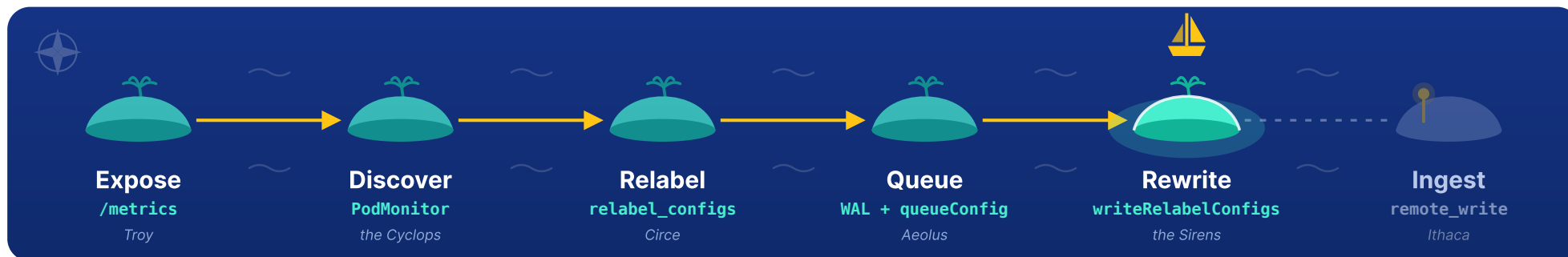
Server mode: samples append to the local TSDB's WAL before remote write delivers them. **Agent mode:** a specialised WAL, no queryable local TSDB.

`remote_write` tails the WAL asynchronously, batches, and retries recoverable failures. It adjusts shards between `min_shards` and `max_shards` — but more shards only help when Prometheus *and* the endpoint have spare capacity.

Delivery lag is the primary health signal — see left.

Prometheus can keep scraping while the remote endpoint is unavailable. It catches up only if unsent samples remain in the WAL and its send capacity eventually exceeds the incoming sample rate.

A keep-list drops most metrics before they are sent



the keep-list and the two routing labels: where the bill is decided

```
writeRelabelConfigs:
- targetLabel: data_stream_dataset
  replacement: k8s
- targetLabel: data_stream_namespace
  replacement: talk
- action: keep # minimal ingestion profile
  sourceLabels: [__name__]
  regex: "up|node_cpu_seconds_total|
  container_memory_working_set_bytes|..."
```

1,644

metric names
held locally

40

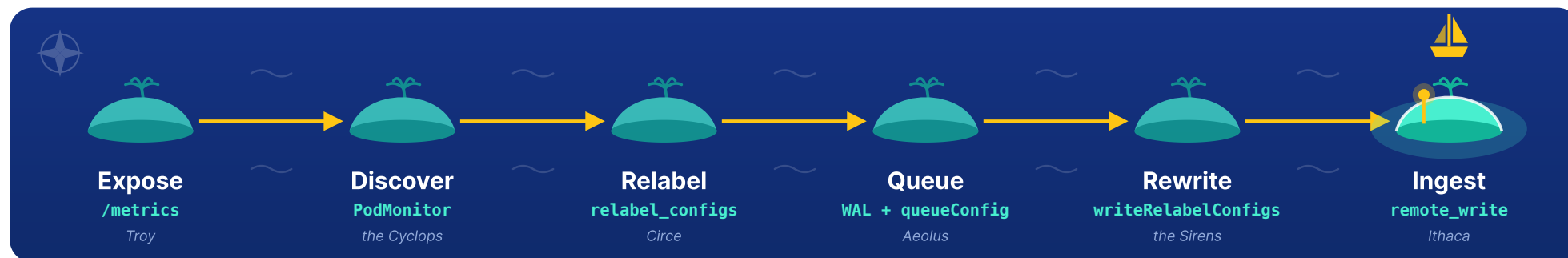
metric names
reaching Elastic

Measured on the cluster behind this talk. AKS ships this idea by default and calls it the **minimal ingestion profile**.

98% dropped at the door — and everything is still queryable locally. Allow-list, never deny-list: a new noisy exporter should not be able to opt itself in.

learn.microsoft.com/azure/azure-monitor/containers/prometheus-metrics-scrape-default#metrics-collected-from-default-targets

What Elastic does with the metric on arrival



labels.* become
TSDS dimensions

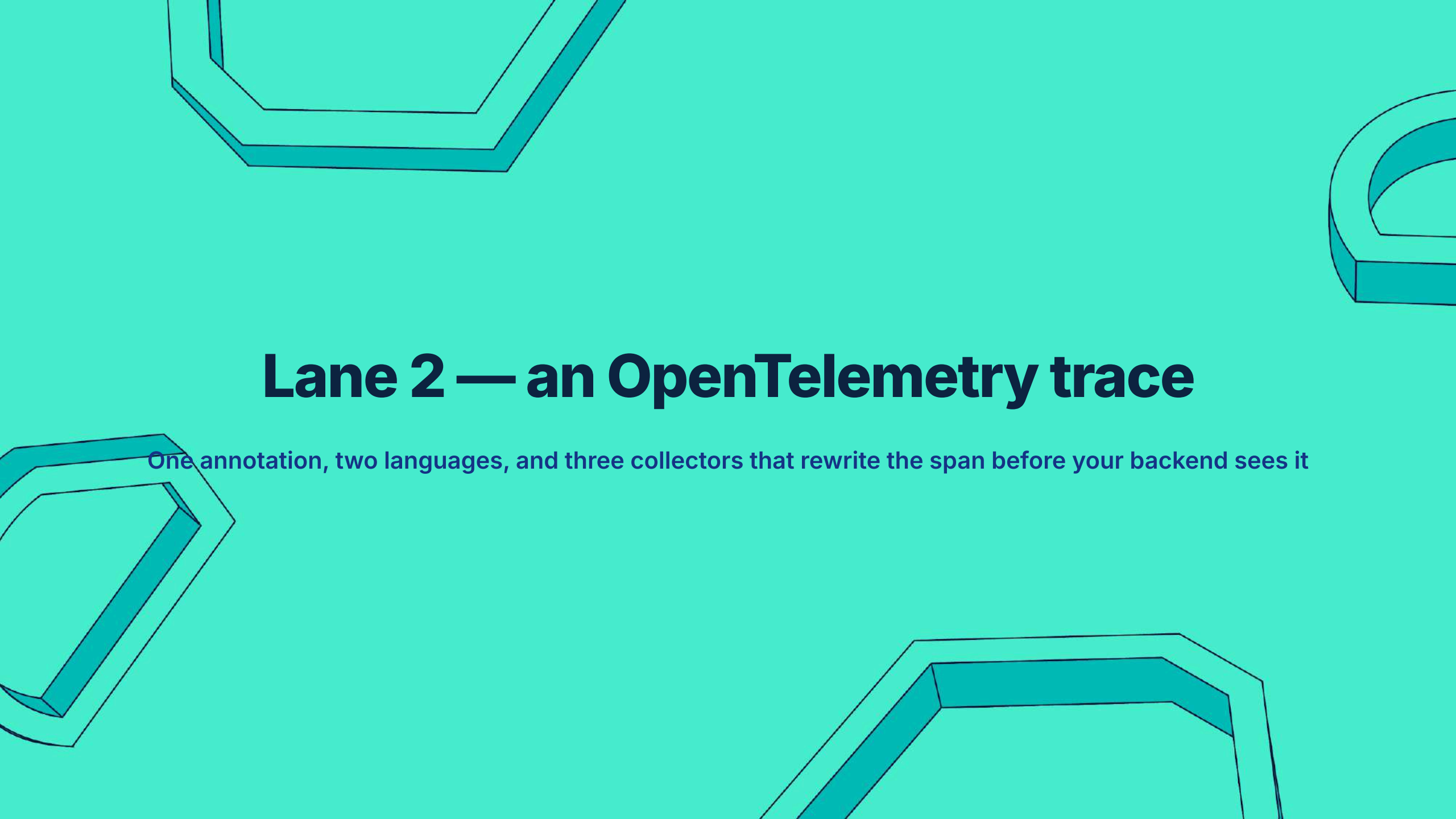
```
remote_write:
- url: https://<project>.ingest.<region>.<csp>
  .elastic.cloud/api/v1/write
  authorization:
    type: ApiKey
    credentials: <key>
```

Every label — including `__name__` — is stored under `labels.*` and marked `time_series_dimension: true`. Values go to `metrics.<name>`.

Two constraints worth knowing: **remote write v1 only**, and NaN/Inf samples plus staleness markers are silently dropped.

Those two routing labels from hop 5 chose the destination: `metrics-k8s.prometheus-talk`. Without them, everything lands in one generic stream.

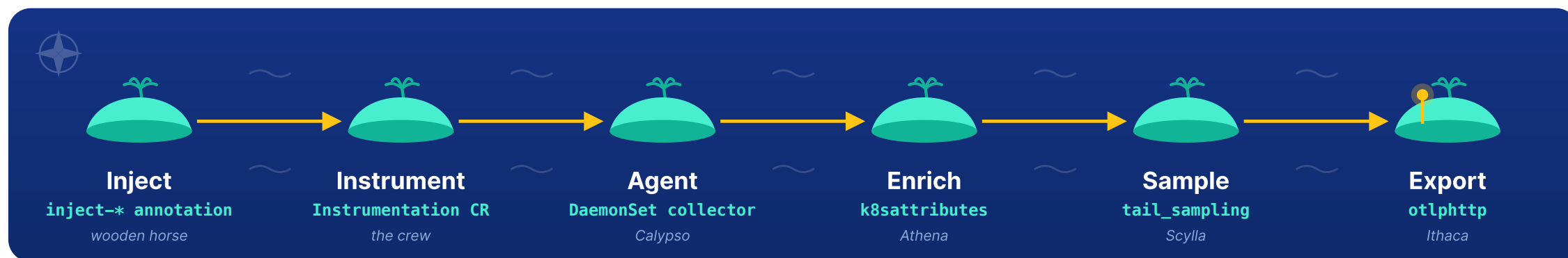
elastic.co/docs/reference/opentelemetry/managed-inputs/prometheus-remote-write



Lane 2 — an OpenTelemetry trace

One annotation, two languages, and three collectors that rewrite the span before your backend sees it

The six steps a trace takes



Note where the two lanes differ: Prometheus *pulls*, and rewrites at the source. OpenTelemetry *pushes*, and every rewrite happens in a collector you operate.

One annotation installs the tracing agent for you



a mutating webhook rewrites the pod spec; the image never changes

```
# checkout
instrumentation.opentelemetry.io/inject-python: "true"
```

```
initContainer  opentelemetry-auto-instrumentation-python
PYTHONPATH    /otel-auto-instrumentation-python/...
```

```
# inventory
instrumentation.opentelemetry.io/inject-java: "true"
```

```
initContainer  opentelemetry-auto-instrumentation-java
JAVA_TOOL_OPTIONS -javaagent:/.../javaagent.jar
```

A mutating webhook rewrites the pod spec at admission. The image never changes. Two gotchas: the **Instrumentation CR** must exist *before* the pod, and running pods must be restarted.

opentelemetry.io/docs/platforms/kubernetes/operator/automatic

What the tracing agent instruments, with no code changes



exporter endpoint, propagators
and the head sampler

checkout	Server	GET /checkout	← FastAPI
checkout	Client	GET	← requests
inventory	Server	GET /inventory/reserve	← Spring MVC
inventory	Client	shop	← JDBC conn
inventory	Client	SELECT shop.stock	← the SQL

One real trace from the demo cluster. **Zero lines of tracing code** in either service.

The `Instrumentation` CR sets the exporter endpoint, the propagators that carry `traceparent` across the HTTP call, and the head sampler.

Keep head sampling at 100%. Anything dropped here is gone before `tail_sampling` at hop 5 could have made a smarter choice.

opentelemetry.io/docs/languages — see each language's supported-libraries list

Why every node runs its own collector



node-local: the span has not
crossed the network yet

```
exporter:  
  endpoint: http://$(K8S_NODE_IP):4318
```

```
mode: daemonset  
ports:  
  otlp-http: {hostPort: 4318}
```

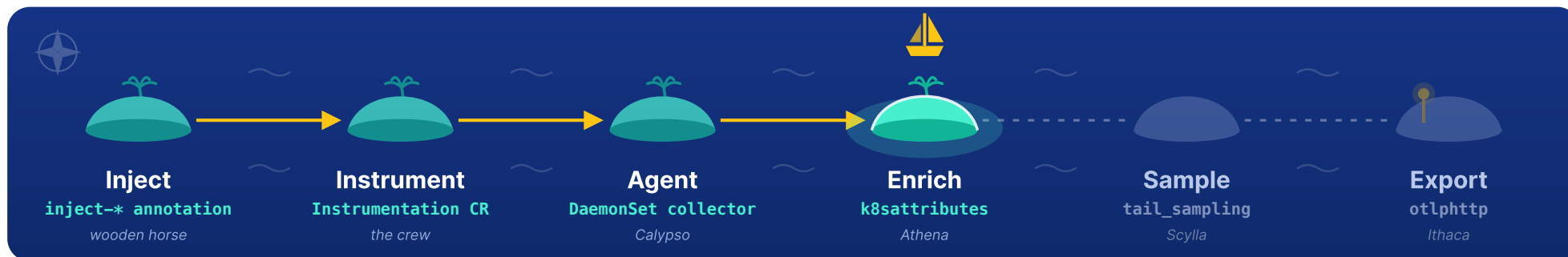
Apps ship to **their own node's** agent via the downward API. The span has not crossed the network yet.

That matters because the next hop needs to ask "which pod sent this", and a node-local process can answer that from a **watch scoped to its own node** instead of every pod in the cluster.

This tier is also your blast radius control: a bad config reaches one node's telemetry, not the whole cluster's.

opentelemetry.io/docs/collector/deployment/agent

Adding Kubernetes metadata to a span



pod, node and labels,
from the API server

```
k8s_attributes:
  extract:
    labels:
      - from: pod
        key: tier
        tag_name: k8s.pod.label.tier
  pod_association:
    - sources: [{from: resource_attribute,
                  name: k8s.pod.ip}]
```

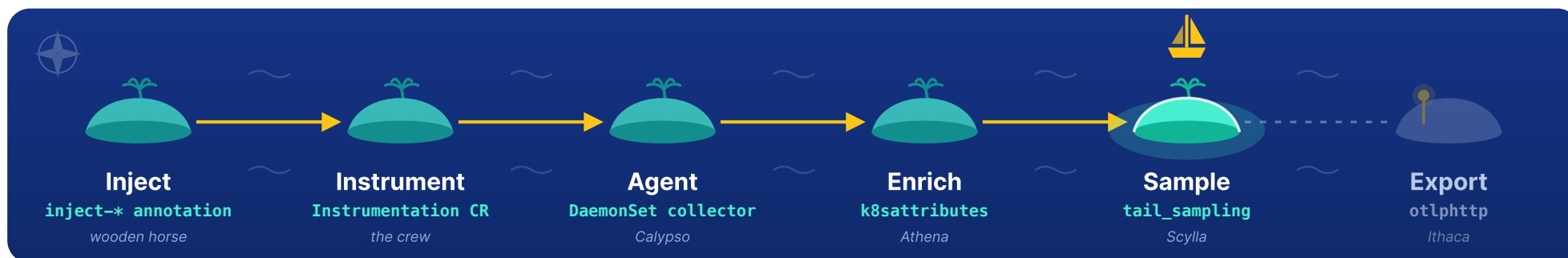
The Operator **already** injected `k8s.pod.name`, `k8s.node.name` and `k8s.deployment.name` via `OTEL_RESOURCE_ATTRIBUTES`.

So when `k8s_attributes` fails, **the spans still look enriched**. In this demo it was doing nothing: `hostPort` traffic is source-NAT'd, so association by connection could not resolve the sender.

Verify with something only the processor can add — pod labels. If `k8s.pod.label.*` is missing, your association is broken no matter how good `k8s.pod.name` looks.

github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/processor/k8sattributesprocessor

Tail sampling decides which traces to keep



whole traces, one process,
one decision

```
tail_sampling:
  decision_wait: 5s
  policies:
    - name: keep-all-errors
      type: status_code
      status_code: {status_codes: [ERROR]}
    - name: sample-the-rest
      type: probabilistic
      probabilistic: {sampling_percentage: 10}
```

Tail sampling needs **every span of a trace in one process**, so it cannot live on a per-node DaemonSet — and the managed OTLP endpoint does not offer it either.

Measured on the demo cluster: **16 traces kept, 170 dropped** against a configured 10%.

One replica here is deliberate. Scale past 1 without a trace-aware `loadbalancing` exporter in front and each replica samples on a partial view of every trace.

What Elastic does with the trace on arrival



```
otlphttp/elastic:  
  endpoint: ${env:ELASTIC_OTLP_HTTP}  
  headers:  
    Authorization: ApiKey ${env:KEY}  
  sending_queue: {enabled: true}  
  retry_on_failure: {enabled: true}
```

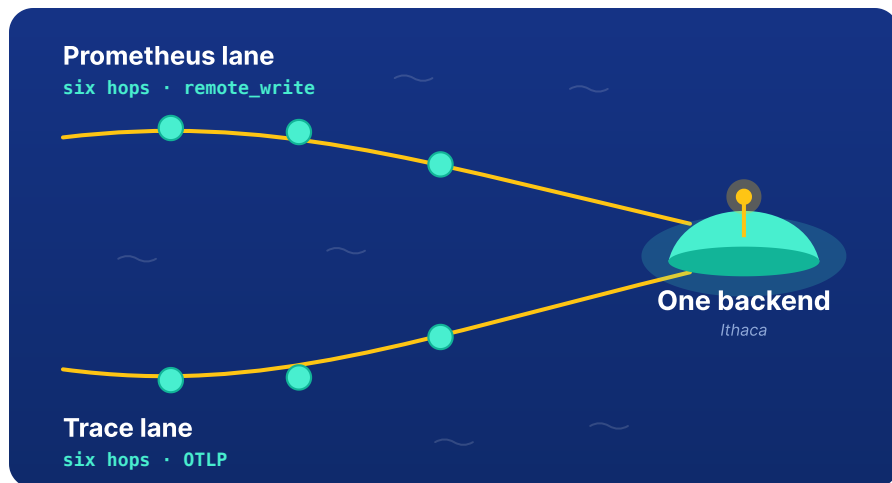
The managed OTLP endpoint speaks **OTLP/HTTP** as well as **gRPC** — worth knowing, because the console hands you a `:443` host that reads like gRPC only.

Spans land in `traces-generic.otel-default` unless you set `data_stream.dataset` yourself.

The API key needs `event:write` on the `apm` application. A general-purpose Elasticsearch key returns `403 PermissionDenied`, which reads exactly like a networking problem.

elastic.co/docs/reference/opentelemetry/managed-inputs/managed-otlp-endpoint

The same pod can send the same data twice



The same container's CPU can reach the same backend **twice** — once as `container_cpu_usage_seconds_total`, once as `k8s.pod.cpu.utilization` if you also run the collector's kubelet receiver.

	Prometheus lane	Trace lane
identity	<code>labels.pod</code>	<code>k8s.pod.name</code>
query	PromQL	ES QL
shape	flat labels	resource attributes

Resiliency differs too. Prometheus replays its **WAL** after an outage. A collector's `sending_queue` is in memory unless you back it with the `file_storage` extension — restart the pod and whatever it held is gone.

Duplication is a choice, not an accident. Pay it deliberately when you want PromQL and recording rules *and* trace correlation. Detect it by asking which lane a number came from — if you cannot answer that, you are already paying twice.

Logs travel the trace lane too: a `filelog` receiver on the same DaemonSet collector puts them through hops 3 to 6 unchanged.

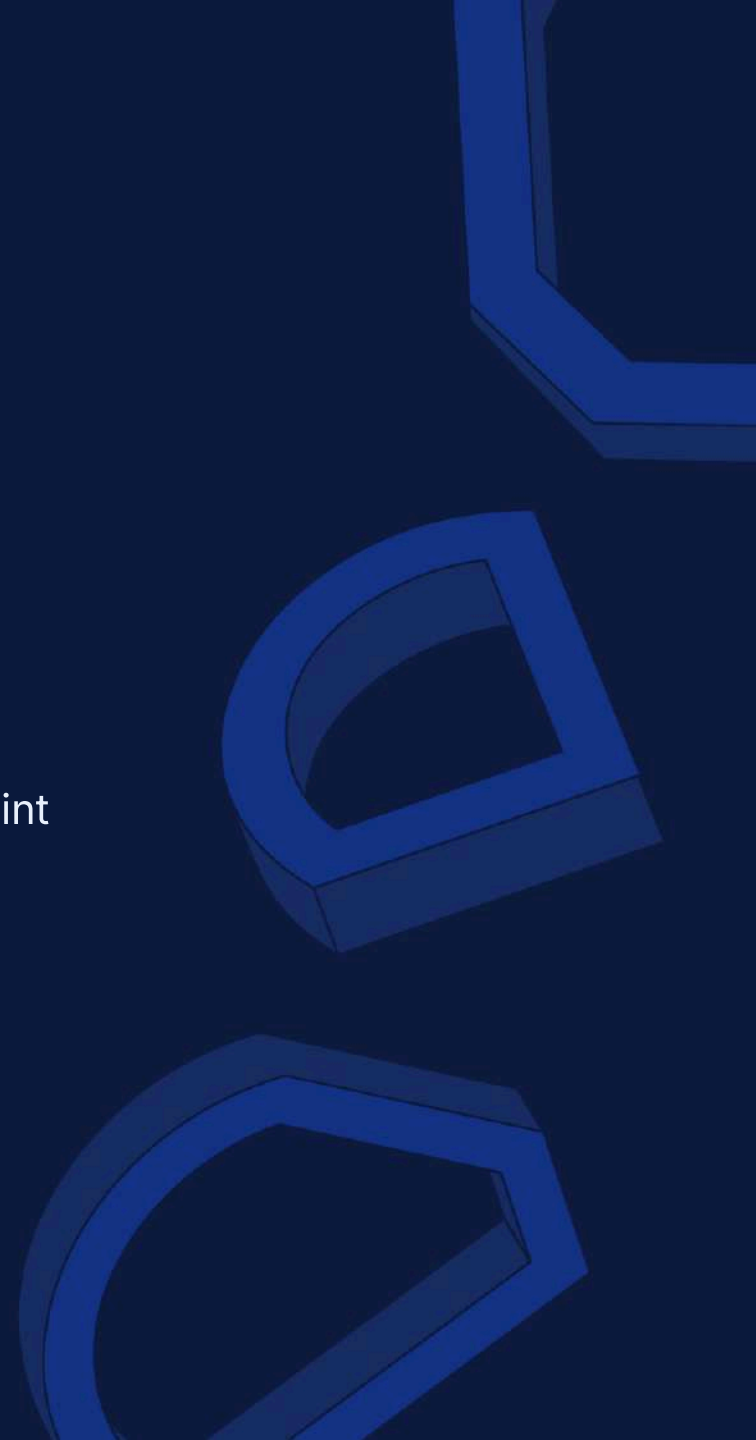
Four settings that control cost and detail

Lever	Prometheus lane	Trace lane	Who owns it
Identity — what the data is called	<code>relabel_configs</code> at scrape (3)	<code>k8s_attributes</code> at the agent (4)	Platform
Volume — what you pay for	keep-list in <code>writeRelabelConfigs</code> (5)	<code>tail_sampling</code> at the gateway (5)	Platform, with a budget
Fidelity — what you can still answer	scrape interval, histogram buckets (1, 3)	head sampling in the <code>Instrumentation</code> CR (2)	Application team
Destination — where it lands	<code>data_stream_dataset</code> / <code>_namespace</code> (5)	<code>data_stream.dataset</code> attribute (6)	Platform

The lever you reach for first is almost always hop 5: `writeRelabelConfigs` on one lane, `tail_sampling` on the other. Both are the last place the data is still yours.

Live demo

k3s on a laptop → twelve hops → a real managed endpoint



What you're about to see

1 · The injection nobody wrote

`kubectl get pod -o yaml` — an init container and a `-javaagent` flag that exist because of one annotation.

2 · Enrichment, proven

A span in Kibana carrying `k8s.pod.label.*` — the attribute only the collector can add.

3 · Discovery

Prometheus targets, and the 1,573-line job a 12-line PodMonitor became.

4 · The keep-list, in numbers

1,644 metric names in Prometheus. 40 in Elastic. Same cluster, same minute.

5 · One pod, two pipelines

The same workload, arriving twice, by two completely different paths.

6 · If the clock allows — OpAMP

Push 10% → 100% sampling to a running collector. The pod's restart count stays at 0.

Clone it, run it, break it

github.com/ericmustin/prometheus-k8s-otel-elastic-talk

```
make up && make images && make deploy && make verify
```

k3s · Prometheus Operator · OpenTelemetry Operator · OTel Collector · OpAMP — all OSS